

Essay Questions

1. What is the default value assigned to array elements in C#?

The default value depends on the data type of the array elements.

- Numeric types such as "int", "double", and "float" → "0"
- "bool" → "false"
- Reference types such as "string" and objects → "null"
- "char" → ""\0""

For example:

```
int[] numbers = new int[3];
```

The elements will initially be

0 0 0

2. What is the difference between Array.Clone() and Array.Copy()?

Both methods are used to copy arrays, but they work differently.

- "Array.Clone()" creates a new array containing copies of all elements.
- "Array.Copy()" copies elements from one array to another existing array and allows you to specify the starting positions and number of elements.

Example:

```
int[] arr1 = { 1, 2, 3 };
```

```
int[] arr2 = (int[])arr1.Clone();
```

"Clone()" creates a new array with the same elements.

With "Array.Copy()":

```
int[] arr2 = new int[3];
```

```
Array.Copy(arr1, arr2, 3);
```

The destination array must already exist.

3. What is the difference between GetLength() and Length for multi-dimensional arrays?

"Length" returns the total number of elements in all dimensions.

"GetLength(dimension)" returns the number of elements in a specific dimension.

Example:

```
int[,] matrix = new int[3, 4];
```

Then:

```
matrix.Length
```

returns:12

While:

```
matrix.GetLength(0)
```

returns:3

and:

```
matrix.GetLength(1)
```

returns:4

So, "Length" gives the total number of elements, while "GetLength()" gives the size of a specific dimension.

4. What is the difference between `Array.Copy()` and `Array.ConstrainedCopy()`?

Both methods copy elements from one array to another.

"`Array.Copy()`" starts copying and may leave the destination array partially modified if an exception occurs during the operation.

"`Array.ConstrainedCopy()`" guarantees that if the entire copy cannot be completed, the destination array remains unchanged.

Therefore, "`Array.ConstrainedCopy()`" provides stronger guarantees when the copy operation must be completed safely.

5. Why is `foreach` preferred for read-only operations on arrays?

"`foreach`" is preferred for read-only operations because it is simple, readable, and does not require managing an index.

Example:

```
foreach (int number in numbers)
{
```

```
    Console.WriteLine(number);  
}
```

It automatically goes through every element of the array.

It also reduces the possibility of index-related errors such as "IndexOutOfRangeException".

6. Which loop (for or foreach) is more efficient for calculating the sum of an array, and why?

For normal array iteration, both "for" and "foreach" have very similar performance.

"for" can be slightly more direct because it uses an index, while "foreach" is usually easier to read and write.

For example:

```
for (int i = 0; i < numbers.Length; i++)  
{  
    sum += numbers[i];  
}
```

and:

```
foreach (int number in numbers)  
{  
    sum += number;  
}
```

Both have $O(n)$ time complexity.

For simple read-only operations, "foreach" is usually preferred because of its readability.

7. Why is input validation important when working with user inputs?

Input validation is important because users may enter invalid, unexpected, or incorrect data.

Validation helps prevent:

- Program crashes
- Exceptions
- Incorrect results
- Invalid data

For example, "int.TryParse()" can safely check whether the user's input can be converted to an integer without throwing an exception.

```
if (int.TryParse(input, out int number))  
{  
    // Valid integer
```

```
}
```

Therefore, input validation makes programs safer, more reliable, and user-friendly.

8. How can you format the output of a 2D array for better readability?

A 2D array can be displayed in a matrix format using nested loops.

The outer loop is used for rows, and the inner loop is used for columns.

For example:

```
for (int i = 0; i < matrix.GetLength(0); i++)  
{  
    for (int j = 0; j < matrix.GetLength(1); j++)  
    {  
        Console.Write(matrix[i, j] + "\t");  
    }  
    Console.WriteLine();  
}
```

Using "\t" adds spacing between columns and makes the output easier to read.

9. When should you prefer a switch statement over if-else?

A "switch" statement is preferred when you need to compare one variable against multiple fixed values.

For example, when converting a month number to a month name:

```
switch (month)  
{  
    case 1:  
        Console.WriteLine("January");  
        break;  
  
    case 2:  
        Console.WriteLine("February");  
        break;  
  
    default:  
        Console.WriteLine("Invalid month");  
        break;  
}
```

A "switch" is usually more readable and organized than a long chain of "if-else" statements when there are many possible fixed cases.

10. What is the time complexity of `Array.Sort()`?

The time complexity of "`Array.Sort()`" is generally $O(n \log n)$.

Here, "n" represents the number of elements in the array.

Therefore, sorting a larger array requires more time as the number of elements increases.

11. What happens if the user enters a value outside the range of 1 to 7 when converting an integer to the `DayOfWeek` enum?

An enum can technically contain an integer value that is not explicitly defined as one of its named members.

For example:

```
DayOfWeek day = (DayOfWeek)8;
```

The conversion itself does not necessarily throw an exception. The value can exist as an enum value even though "8" is not one of the defined days.

Therefore, we should validate the input before converting it:

```
if (number >= 1 && number <= 7)
{
    DayOfWeek day = (DayOfWeek)Enum.Parse(
        typeof(DayOfWeek),
        number.ToString()
    );
}
else
{
    Console.WriteLine("Invalid input.");
}
```

This ensures that only valid values from "1" to "7" are accepted.

Additional Essay Questions: Bonus

12. What is the difference between a shallow copy and a deep copy?

A shallow copy creates another reference to the same array, so both variables refer to the same array in memory.

Example:

```
int[] arr1 = { 1, 2, 3 };  
int[] arr2 = arr1;
```

If we modify "arr2", "arr1" will also be affected.

A deep copy creates a separate array with its own memory.

Example:

```
int[] arr2 = (int[])arr1.Clone();  
Modifying "arr2" will not affect "arr1" for an array of value types such as "int".
```

13. What is an IndexOutOfRangeException?

"IndexOutOfRangeException" occurs when we try to access an array element using an index that is outside the valid range.

For example:

```
int[] numbers = { 10, 20, 30 };
```

```
Console.WriteLine(numbers[5]);
```

The valid indexes are "0", "1", and "2", so accessing index "5" causes an "IndexOutOfRangeException".

14. What is the difference between a one-dimensional array and a two-dimensional array?

A one-dimensional array stores elements in a single sequence.

Example:

```
int[] numbers = { 1, 2, 3, 4 };
```

A two-dimensional array stores elements using rows and columns.

Example:

```
int[,] matrix =
```

```
{  
    { 1, 2 },  
    { 3, 4 }  
};
```

A 1D array uses one index, while a 2D array uses two indexes

15. What is the purpose of `Array.Clear()`?

"`Array.Clear()`" sets a range of elements in an array to their default values. For an "int" array, the elements become "0".

Example:

```
int[] numbers = { 1, 2, 3 };
```

```
Array.Clear(numbers, 0, numbers.Length);
```

After clearing:

```
0 0 0
```

16. What is the purpose of `Array.Reverse()`?

"`Array.Reverse()`" reverses the order of the elements in an array.

Example:

```
int[] numbers = { 1, 2, 3, 4 };
```

```
Array.Reverse(numbers);
```

The result is: 4 3 2 1

17. What is the purpose of `Array.IndexOf()` and `Array.LastIndexOf()`?

"`Array.IndexOf()`" returns the index of the first occurrence of a specified value.

"Array.LastIndexOf()" returns the index of the last occurrence of that value.

Example:

```
int[] numbers = { 5, 8, 2, 8, 10 };  
Array.IndexOf(numbers, 8);  
returns: 1
```

while:

```
Array.LastIndexOf(numbers, 8);
```

returns: 3

18. What is an enum in C#?

An enum, or enumeration, is a value type used to define a set of named constants.

For example:

```
enum DayOfWeek  
{  
    Monday = 1,  
    Tuesday,  
    Wednesday,  
    Thursday,  
    Friday,  
    Saturday,  
    Sunday  
}
```

Enums make code easier to read and understand because we can use meaningful names instead of numeric values.